MyCodeSchool

Exerciții — șiruri de caractere (4)

Python · proiectul `py-siruri` · Luca

Ce ai voie: tot din fișele 1, 2 și 3 · felii · `for litera in text`, `for i in range(...)` cu `text[i]` și `text[i - 1]` · `.split()`, `.split(",")`, `.split("-")` · contor, acumulator, maxim/minim · `in`, `not in` · `int()`, `str()`, `len()`, f-string · `.lower()`, `.isdigit()`, `.find()` · dicționare cu `.items()` · tupluri · liste cu `.append()` și `[x for x in ... if ...]`.
Nicio noțiune nouă în fișa asta.
Încă NU folosim: `max()`, `min()`, `sum()`, `sorted()`, `.join()`, `while`, `try` / `except`, `print` **în funcții**.

Cum lucrezi

Scrii în `siruri4.py`, fiecare funcție întoarce cu `return`, iar `main.py` afișează. Un `print` pentru **fiecare rând** din tabel; ieșirea e identică cu coloana „întoarce”:

```
from siruri4 import sold_minim

def main():
    print(sold_minim("100,-300,50"))
    print(sold_minim("500,200"))

if __name__ == "__main__":
    main()
```

Ieșire așteptată: `-200` și, pe rândul următor, `500`. Pentru exercițiul cu dicționar, apelul arată așa: `extreme_pe_zile({"luni": 5, "marti": -2})`.

La clasă

Soldul cel mai mic din cont

1

```
def sold_minim(miscari: str) -> int:
```

Mișcările din cont sunt scrise separate prin virgulă: `100` înseamnă depunere, `-300` retragere. Contul pornește gol. După fiecare mișcare ai un sold. Întoarce **cel mai mic sold** pe care l-a avut contul. Fără mișcări: `0`.

apel	întoarce
`sold_minim("100,-300,50")`	`-200`
`sold_minim("500,200")`	`500`
`sold_minim("-50")`	`-50`
`sold_minim("-20,-30,-10")`	`-60`
`sold_minim("300,-100,-100")`	`100`
`sold_minim("2000")`	`2000`
`sold_minim("")`	`0`

Al câtelea e cel mai lung nume?

2

```
def pozitia_celui_mai_lung(nume: str) -> int:
```

Numele sunt separate prin virgulă. Întoarce **al câtelea** nume e cel mai lung (primul nume e al 1-lea). La lungimi egale, cel de **mai devreme**. Fără nume: `0`.

apel	întoarce
`pozitia_celui_mai_lung("Ana,Maria,Ion")`	`2`
`pozitia_celui_mai_lung("Ion,Ana,Dan")`	`1`
`pozitia_celui_mai_lung("Mara,Ioan,Alexandru")`	`3`
`pozitia_celui_mai_lung("Ana,Alexandru")`	`2`
`pozitia_celui_mai_lung("Ana")`	`1`
`pozitia_celui_mai_lung("")`	`0`

Data scrisă ca la noi 3

```
def data_romaneasca(data_iso: str) -> str:
```

Primești `an-luna-zi` (ziua și luna pe două cifre) și întorci `zi.luna.an`, **fără** zero în față la zi și la lună. Pentru `""` întoarce `""`.

apel	întoarce
`data_romaneasca("2026-10-21")`	`"21.10.2026"`
`data_romaneasca("2026-03-05")`	`"5.3.2026"`
`data_romaneasca("2027-12-01")`	`"1.12.2027"`
`data_romaneasca("2026-01-31")`	`"31.1.2026"`
`data_romaneasca("")`	`""`

Mai grele

Ziua cea mai slabă și ziua cea mai bună 4

```
def extreme_pe_zile(profit: dict) -> tuple:
```

Dicționarul are ca cheie numele zilei și ca valoare profitul din ziua respectivă (negativ = pierdere). Întoarce, **în ordinea asta**: ziua cu cel mai mic profit, acel profit, ziua cu cel mai mare profit, acel profit. La egalitate, ziua care apare **prima** în dicționar. Dicționar gol: `("", 0, "", 0)`.

apel	întoarce
`extreme_pe_zile({"luni": 5, "marti": -2, "miercuri": 9})`	`("marti", -2, "miercuri", 9)`
`extreme_pe_zile({"luni": -4, "marti": -9, "miercuri": -1})`	`("marti", -9, "miercuri", -1)`
`extreme_pe_zile({"luni": 1500, "marti": 1200})`	`("marti", 1200, "luni", 1500)`
`extreme_pe_zile({"luni": 12, "marti": 12})`	`("luni", 12, "luni", 12)`
`extreme_pe_zile({"duminica": -7})`	`("duminica", -7, "duminica", -7)`
`extreme_pe_zile({})`	`("", 0, "", 0)`

Diferența dintre cel mai mare și cel mai mic 5

```
def diferenta_maxima(numere: str) -> int:
```

Numerele (pot fi negative) sunt separate prin virgulă. Întoarce **cel mai mare minus cel mai mic**. Fără numere: `0`.

apel	întoarce
`diferenta_maxima("3,10,4")`	`7`
`diferenta_maxima("-5,-9,-2")`	`7`
`diferenta_maxima("1500,2000,1800")`	`500`
`diferenta_maxima("5,1")`	`4`
`diferenta_maxima("7,7")`	`0`
`diferenta_maxima("42")`	`0`
`diferenta_maxima("-3")`	`0`
`diferenta_maxima("")`	`0`

Unde apare prima cifră? 6

```
def pozitia_primei_cifre(text: str) -> int:
```

Întoarce **al câtelea caracter** din text este prima cifră (primul caracter e al 1-lea). Dacă textul nu are nicio cifră: `0`.

apel	întoarce
`pozitia_primei_cifre("Bloc 5, ap. 12")`	`6`
`pozitia_primei_cifre("abc9")`	`4`
`pozitia_primei_cifre("a1b2")`	`2`
`pozitia_primei_cifre("7abc")`	`1`
`pozitia_primei_cifre("9")`	`1`
`pozitia_primei_cifre("Strada Libertatii")`	`0`
`pozitia_primei_cifre("")`	`0`

Provocări

Fără indicii.

Locul doi la concurs 7

```
def a_doua_cea_mai_mare(numere: str) -> int:
```

Numerele (pot fi negative) sunt separate prin virgulă. Întoarce a **doua cea mai mare valoare**, numărând fiecare număr din text: dacă cel mai mare apare de două ori, locul doi e tot el. Cu mai puțin de două numere: `0`.

apel	întoarce
`a_doua_cea_mai_mare("5,9,2")`	`5`
`a_doua_cea_mai_mare("9,5,2")`	`5`
`a_doua_cea_mai_mare("2,5,9")`	`5`
`a_doua_cea_mai_mare("5,9,9")`	`9`
`a_doua_cea_mai_mare("-5,-3,-8")`	`-5`
`a_doua_cea_mai_mare("1500,2000,1800")`	`1800`
`a_doua_cea_mai_mare("4,8")`	`4`
`a_doua_cea_mai_mare("8,4")`	`4`
`a_doua_cea_mai_mare("7")`	`0`
`a_doua_cea_mai_mare("")`	`0`

Cea mai lungă serie care crește 8

```
def cea_mai_lunga_crestere(numere: str) -> tuple:
```

Numerele (pot fi negative) sunt separate prin virgulă. O serie care crește are fiecare număr **strict mai mare** decât cel dinainte. Întoarce `(început, lungime)` pentru cea mai lungă serie: **al câtelea** număr o începe (primul e al 1-lea) și câte numere are. La lungimi egale, seria de **mai devreme**. Fără numere: `(0, 0)`.

apel	întoarce
`cea_mai_lunga_crestere("1,2,3,1,2,3,4,0")`	`(4, 4)`
`cea_mai_lunga_crestere("1,2,3,4")`	`(1, 4)`
`cea_mai_lunga_crestere("3,1,2")`	`(2, 2)`
`cea_mai_lunga_crestere("-5,-3,-1,-8")`	`(1, 3)`
`cea_mai_lunga_crestere("1,2,1,2")`	`(1, 2)`
`cea_mai_lunga_crestere("5,4,3")`	`(1, 1)`
`cea_mai_lunga_crestere("2,2,2")`	`(1, 1)`
`cea_mai_lunga_crestere("7")`	`(1, 1)`
`cea_mai_lunga_crestere("")`	`(0, 0)`

Numerele peste medie 9

```
def peste_medie(numere: str) -> list:
```

Numerele (pot fi negative) sunt separate prin virgulă. Întoarce **lista** numerelor **strict mai mari** decât media lor, în ordinea din text. Fără numere: lista goală.

apel	întoarce
`peste_medie("1,2,3,10")`	`[10]`
`peste_medie("4,9,1,8")`	`[9, 8]`
`peste_medie("-3,-1,-8")`	`[-3, -1]`
`peste_medie("1500,100,1600")`	`[1500, 1600]`
`peste_medie("5,5")`	`[]`
`peste_medie("7")`	`[]`
`peste_medie("")`	`[]`

Înainte să zici gata, la fiecare funcție:
1. Un `print` pentru **fiecare rând** din tabel; ieșirea e identică cu „întoarce”. Fără `None`.
2. Încearcă fiecare funcție și pe câteva cazuri alese de tine, care nu sunt în tabel.